Dar es Salaam Smart Daladala Passenger & Fare System

Project Type: Public Transport Passenger Journey, Routing and Fare Platform
Primary Framework: Django
Database: PostgreSQL + PostGIS
Frontend: HTML, CSS, JavaScript
Backend API: Django REST Framework
Geospatial Processing: GeoDjango + PostGIS
Routing: Python graph/routing engine
Target Location: Dar es Salaam, Tanzania

---

1. Project Purpose

The system is designed to improve the passenger experience of using daladala public transport in Dar es Salaam.

The system will help passengers:

1. Find a route from their current location to a destination.
2. Determine whether they need to change daladala.
3. Identify where to board and where to get off.
4. Track their journey.
5. Receive a reminder when approaching their destination.
6. Determine the appropriate fare for their journey.
7. Identify themselves to an existing digital fare/payment system.
8. Eventually allow automatic fare calculation based on boarding and alighting locations.
9. Reduce the amount of manual work required from conductors.

The project is designed to integrate with an existing team's digital payment/card system rather than replacing it.

---

2. Problem Being Solved

The current daladala experience creates several passenger and operator problems.

Passengers may:

- not know which daladala to take;
- not know where to change vehicles;
- not know the correct stop;
- be unfamiliar with Dar es Salaam;
- fall asleep and miss their destination;
- ask conductors to remind them;
- struggle to know the appropriate fare;
- forget to scan when leaving a vehicle.

Conductors may:

- have to manually process many passengers;
- receive repeated destination/reminder requests;
- manually enter fare amounts in some digital payment systems;
- make accidental input errors in crowded and moving vehicles;
- potentially enter incorrect amounts intentionally or accidentally.

The proposed system moves appropriate responsibilities from the conductor to software.

---

3. Core Concept

The system has two major parts.

Part A: Passenger Journey Layer

This project develops:

- transport data;
- route planning;
- destination search;
- transfer planning;
- journey tracking;
- destination reminders;
- boarding/alighting detection;
- fare calculation.

Part B: Existing Payment System

A separate team already has:

- passenger cards/QR identification;
- digital wallet/payment functionality;

- payment processing.

The two systems should communicate through APIs.

The payment system remains responsible for actual financial transactions.

---

4. Scope

Included

The system will include:

- Dar es Salaam daladala stops;
- routes;
- route sequences;
- route geographic paths;
- station coordinates;
- station aliases;
- transfer points;
- fare rules;
- passenger accounts;
- journey planning;
- journey tracking;
- vehicle tracking;
- destination reminders;
- boarding records;
- alighting records;
- automatic alighting estimation;
- fare calculation;
- payment-system integration;
- administrative management;
- transport-data management;
- system monitoring.

Not Initially Included

The first production version will NOT attempt to build:

- a replacement for the existing payment platform;
- a complete banking system;
- a new physical payment card;
- a physical fare machine;

- full traffic prediction;
- autonomous vehicle tracking hardware;
- a nationwide transport system;
- every Tanzanian city;
- advanced AI route prediction.

These may become future projects.

---

5. Users

5.1 Passenger

A passenger can:

- create/login to an account;
- view their QR identifier;
- find their current location;
- search for a destination;
- plan a journey;
- view routes;
- view transfers;
- view estimated fare;
- start a journey;
- view current journey status;
- receive destination reminders;
- manually confirm alighting;
- view journey history;
- view payment/fare information.

5.2 Conductor

The conductor's interaction should be minimal.

Depending on integration with the existing payment system, the conductor may:

- scan passenger QR;
- identify passenger;
- identify the current vehicle/trip;
- receive confirmation of boarding/payment;
- optionally confirm exceptional events.

The conductor should NOT normally have to manually type the fare.

5.3 Vehicle/Operator

An operator can:

- register vehicles;
- assign vehicles to routes;
- view vehicle activity;
- view journey/payment summaries;
- monitor operational information.

5.4 Administrator

An administrator can manage:

- stations;
- routes;
- route-stop relationships;
- route geometry;
- fares;
- vehicles;
- operators;
- users;
- aliases;
- transfer points;
- data sources;
- system configuration.

---

6. Transport Data Model

The transport network is represented as a graph.

6.1 Station

A station represents a recognized passenger boarding/alighting location.

Fields:

- station ID
- canonical name
- aliases
- latitude
- longitude
- ward
- district
- active status
- source
- verification date

6.2 Route

A route represents a daladala service between locations.

Fields:

- route ID
- route name
- origin
- destination
- direction
- active status
- source
- verification date

6.3 Route Stop

This connects a route to a station.

Fields:

- route ID
- station ID
- sequence number

Example:

```
Route A

1. Kimara
2. Ubungo
```

```
3. Mwenge
4. Posta
```

The sequence is essential for journey and fare calculations.

6.4 Route Geometry

Represents the geographic path followed by a route.

Fields:

- route ID
- shape ID
- sequence
- latitude
- longitude
- cumulative distance

This allows the system to determine where a vehicle is along its route.

6.5 Station Alias

Maps alternative names to canonical stations.

Example:

```
Canonical:
Mwenge

Aliases:
Mwenge Bus Stand
Mwenge Stand
```

6.6 Transfer Point

A transfer point identifies where passengers can change routes.

Fields:

- station ID
- route A

- route B
- walking distance
- estimated transfer time
- active status

---

7. Fare System

Fare information must be stored as data rather than hard-coded into application logic.

Fare Rule

Fields:

- fare ID
- passenger category
- minimum distance
- maximum distance
- fare amount
- effective date
- expiry date
- source
- notes

Example:

```
0–10 km     → TSh 600
11–15 km    → TSh 700
16–20 km    → TSh 800
...
```

The actual production values must always be verified against the current authoritative transport authority rules.

---

8. Passenger Journey Planning

A passenger enters:

```
Starting location:
Ubungo

Destination:
```

```
Mbezi
```

The system determines:

1. nearest relevant station;
2. possible routes;
3. direct routes;
4. transfer options;
5. destination station;
6. estimated journey distance;
7. estimated travel time where available;
8. applicable fare.

---

9. Route-Finding Algorithm

The transport network is represented as a graph.

```
Station = Node
Route connection = Edge
```

The routing engine must support:

Direct journey

```
A → B
```

One transfer

```
A → B → C
```

Multiple transfers

```
A → B → C → D
```

The system should calculate multiple route options where appropriate.

Route ranking can consider:

- travel distance;
- estimated travel time;
- number of transfers;
- walking distance;
- route availability.

The system must not assume that the geographically shortest route is always the best route.

---

10. Journey Creation

When the passenger selects a route, a journey is created.

A journey contains:

- journey ID;
- passenger ID;
- planned origin;
- planned destination;
- selected route(s);
- current route;
- journey status;
- start time;
- expected destination;
- actual alighting location;
- calculated fare.

Journey states:

```
PLANNED
BOARDING
IN_PROGRESS
TRANSFER_REQUIRED
COMPLETED
CANCELLED
```

---

11. Passenger Boarding

When a passenger boards a vehicle:

```
Passenger QR
      ↓
Vehicle identified
      ↓
Route identified
      ↓
Boarding recorded
```

The system records:

- passenger ID;

- vehicle ID;
- route ID;
- timestamp;
- GPS location;
- nearest station;
- boarding station.

---

12. Vehicle Tracking

A vehicle may periodically report:

- vehicle ID;
- timestamp;
- latitude;
- longitude;
- speed;
- heading;
- route ID.

Example:

```
Vehicle: V042
Latitude: -6.xxxx
Longitude: 39.xxxx
Speed: 32 km/h
Heading: 180°
Time: 15:42:01
```

Tracking frequency will be determined during testing based on:

- accuracy;
- network usage;
- battery consumption;
- server load.

---

13. Passenger Tracking

With appropriate permission, the passenger's device may provide:

- latitude;
- longitude;

- timestamp;
- GPS accuracy;
- speed.

Continuous second-by-second tracking is NOT required by default.

The system should use efficient/adaptive location updates.

---

14. Passenger/Vehicle Association

After boarding, the system associates:

```
Passenger P001
       +
Vehicle V042
```

The system monitors whether both continue travelling together.

Potential signals:

- geographic distance;
- speed;
- heading;
- route position;
- time;
- GPS accuracy.

---

15. Automatic Alighting Detection

The system attempts to identify when a passenger has left the vehicle.

Example:

```
Before:

Passenger ──────────┐
                    ├── moving together
Vehicle ────────────┘

After:

Passenger ──────── stays near Stop X
```

```
Vehicle ──────────────→ continues moving
```

The system generates:

- probable alighting location;
- probable alighting station;
- confidence score.

Example:

```
Probable stop:
Mwenge

Confidence:
91%
```

Automatic detection is probabilistic.

It must never be treated as infallible.

---

16. Manual Alighting

The passenger can manually confirm alighting.

Methods may include:

- QR scan;
- application button;
- explicit confirmation.

Manual confirmation takes priority over automatic detection.

---

17. Destination Reminder

Passenger selects:

```
Destination:
Mbezi
```

The system monitors their journey.

When approaching the destination:

```
You're approaching Mbezi.
Get ready to get off.
```

Optional future functionality:

```
1 stop away
2 stops away
Approx. 500 metres away
```

The reminder must also work after transfers.

---

18. Automatic Fare Calculation

Once the system has:

```
Boarding point
+
Alighting point
+
Route
```

it determines:

1. travelled route distance;
2. applicable fare band;
3. passenger category;
4. final fare.

Example:

```
Boarding:
Ubungo

Alighting:
Mwenge

Distance:
X km

Fare:
TSh XXX
```

The fare engine must use database-stored fare rules.

---

19. Payment Integration

The system will NOT process the actual wallet transaction.

Instead:

```
Journey System
      ↓
Calculated fare
      ↓
Payment API
      ↓
Existing Payment System
      ↓
Wallet/card deduction
```

The integration should exchange:

- passenger ID;
- journey ID;
- fare;
- transaction reference;
- payment status.

Possible payment states:

```
PENDING
SUCCESS
FAILED
REFUNDED
```

---

20. QR Functionality

The passenger receives a digital QR identity through the application.

The QR identifies the passenger.

The QR should NOT contain sensitive financial information directly.

The system should use a secure identifier/token.

The existing payment team's QR/card system should be reused where possible.

---

21. Offline Considerations

The project should eventually support limited operation during poor connectivity.

Potential offline functionality:

- displaying passenger QR;
- displaying saved destination;
- storing temporary journey events;
- caching station/route data;
- synchronizing events when connectivity returns.

Financial transactions should not be considered successful merely because a device was offline.

Payment synchronization must have a reliable server-side confirmation mechanism.

---

22. Security

The system must include:

- authentication;
- authorization;
- encrypted communication;
- secure QR tokens;
- protection against duplicate transactions;
- protection against replay attacks;
- server-side fare validation;
- audit logs;
- rate limiting;
- secure location-data handling.

Passengers must explicitly grant location permissions.

---

23. Privacy

Location data is sensitive operational information.

The system should:

- collect only necessary location data;
- explain why location is needed;
- allow users to disable tracking where possible;
- restrict access to location history;
- avoid exposing passenger movement publicly;
- define data retention policies.

---

24. Django Architecture

Recommended structure:

```
project/
├── config/
├── accounts/
├── transport/
├── routing/
├── journeys/
├── tracking/
├── fares/
├── payments/
├── notifications/
├── vehicles/
├── api/
└── common/
```

accounts

Users, authentication and permissions.

transport

Stations, routes, route stops, aliases, transfers and route geometry.

routing

Route-finding algorithms.

`journeys`

Passenger journeys, boarding and alighting.

`tracking`

Vehicle and passenger location processing.

`fares`

Fare rules and fare calculation.

`payments`

Integration with the external payment system.

`notifications`

Destination reminders and other passenger notifications.

`vehicles`

Vehicles, operators and assignments.

`api`

REST API endpoints.

---

25. Technology Stack

Backend

Python + Django

API

Django REST Framework

Database

PostgreSQL

Geographic database

PostGIS

Geographic Django support

GeoDjango

Frontend

HTML + CSS + JavaScript initially

Routing

Python graph algorithms, potentially NetworkX during development

Maps

A suitable mapping provider/OpenStreetMap-based solution, selected after evaluating licensing, cost and technical requirements.

Background processing

A task queue such as Celery may be introduced when background tracking/notifications require it.

Deployment

Production deployment will be selected after the MVP architecture is stable.

---

26. Development Phases

The system will be developed in the following order.

PHASE 0 — Requirements and Research

Deliverables

- finalized requirements;

- existing competitor research;
- official fare sources;
- transport-data sources;
- technical feasibility analysis.

Definition of Done

We understand what we are building and what we are NOT building.

PHASE 1 — Transport Data

Build/import:

- stations;
- coordinates;
- aliases;
- routes;
- route-stop sequences;
- route geometry;
- transfer points;
- fare rules.

Definition of Done

The database can represent the Dar es Salaam transport network required for the MVP.

PHASE 2 — Django Foundation

Build:

- Django project;
- PostgreSQL;
- PostGIS;
- environment configuration;
- applications;
- authentication;
- admin interface;
- initial models;
- migrations.

Definition of Done

The application starts successfully and administrators can manage transport data.

PHASE 3 — Passenger Search

Build:

- station search;
- destination search;
- current-location support;
- station aliases;
- map display.

Definition of Done

A passenger can enter:

```
From: Ubungo
To: Mbezi
```

and the system recognizes both locations.

PHASE 4 — Routing Engine

Build:

- transport graph;
- direct route search;
- transfer detection;
- shortest/lowest-cost route;
- route alternatives.

Definition of Done

The system can produce a valid route between supported locations.

PHASE 5 — Passenger Journey

Build:

- journey creation;
- planned route;
- journey status;
- boarding;
- destination;
- journey history.

Definition of Done

A passenger can start and manage a complete simulated journey.

PHASE 6 — Destination Reminder

Build:

- destination monitoring;
- proximity detection;
- stop-based reminders;
- notification system.

Definition of Done

A passenger can select a destination and receive a reminder when approaching it.

PHASE 7 — Vehicle Tracking

Build:

- vehicle location reporting;
- route association;
- vehicle movement;
- location storage;
- map visualization.

Definition of Done

A test vehicle can be tracked along a known route.

PHASE 8 — Automatic Alighting Detection

Build and test:

- passenger/vehicle correlation;
- movement comparison;
- separation detection;
- nearest-stop detection;
- confidence scoring;
- manual override.

Definition of Done

Controlled experiments demonstrate that the system can reasonably identify passenger alighting events.

This feature must be validated with real-world testing before being trusted for financial transactions.

PHASE 9 — Fare Engine

Build:

- route-distance calculation;
- fare-rule lookup;
- passenger category;
- fare calculation;
- fare history;
- fare validation.

Definition of Done

The system consistently calculates the applicable fare for supported journeys using the configured fare rules.

PHASE 10 — Payment Integration

Integrate with the existing team's payment system.

Build:

- passenger identification;
- payment request;
- transaction reference;
- payment confirmation;
- failure handling;
- reconciliation.

Definition of Done

A complete simulated journey can produce a correct payment request and receive a confirmed payment response.

PHASE 11 — Offline Support

Add:

- cached transport data;
- QR availability;
- offline journey events;
- synchronization;
- duplicate-event protection.

Definition of Done

The application can survive temporary connectivity loss without corrupting journey or payment records.

PHASE 12 — Security and Privacy

Perform:

- authentication testing;
- authorization testing;

- API security;
- QR security;
- transaction security;
- location-data protection;
- audit logging.

Definition of Done

Known security requirements have been implemented and tested.

PHASE 13 — Real-World Pilot

Test with a small number of:

- passengers;
- conductors;
- vehicles;
- routes.

Measure:

- route accuracy;
- station accuracy;
- fare accuracy;
- reminder accuracy;
- GPS accuracy;
- alighting detection accuracy;
- payment reliability;
- battery usage;
- network usage;
- user experience.

Definition of Done

The system works reliably enough in a controlled pilot to identify remaining production problems.

PHASE 14 — Integration With Partner Team

Present the working passenger system to the team that developed the payment system.

Demonstrate:

```
Passenger
   ↓
Journey planning
   ↓
Boarding
   ↓
Vehicle tracking
   ↓
Destination detection
   ↓
Fare calculation
   ↓
Payment API
   ↓
Their payment system
```

Determine:

- API compatibility;
- technical ownership;
- intellectual property;
- responsibilities;
- revenue/business model;
- deployment responsibilities.

---

27. Testing Strategy

Testing must occur at several levels.

Unit Tests

Test:

- fare calculations;
- route calculations;
- distance calculations;
- station matching;
- alias matching;
- journey state transitions.

Integration Tests

Test:

- Django ↔ PostgreSQL;
- Django ↔ PostGIS;
- routing ↔ transport data;
- journey ↔ tracking;
- journey ↔ fare engine;
- system ↔ payment API.

Simulation Tests

Create simulated:

- passenger;
- vehicle;
- route;
- GPS movements.

Test automatic alighting without using real passengers.

Real-World Tests

Use controlled journeys in actual daladala environments.

---

28. Success Criteria

The project is considered technically complete when:

Transport

- required Dar es Salaam transport data is loaded;
- stations can be searched;
- routes can be searched;
- transfers can be identified.

Navigation

- passenger can enter origin/destination;
- system can produce a valid route;
- multi-route journeys work.

Journey

- passenger can start a journey;

- boarding is recorded;
- vehicle can be tracked;
- journey status updates correctly.

Reminder

- destination can be selected;
- passenger receives a destination reminder.

Fare

- boarding and alighting can be determined;
- correct fare rule is selected;
- fare is calculated.

Automatic alighting

- system can estimate passenger alighting;
- confidence is recorded;
- manual override works.

Payment

- calculated fare can be passed to the existing payment system;
- payment result is recorded;
- duplicate payments are prevented.

Administration

- administrators can update stations, routes and fares without modifying source code.

Security

- authentication works;
- authorization works;
- payment requests are protected;
- sensitive location information is protected.

---

29. Project Completion Definition

The project is **DONE** when the following complete journey works reliably:

```
Passenger opens application
```

```
        ↓
Chooses current location
        ↓
Chooses destination
        ↓
System finds route
        ↓
System shows:
- route
- stops
- transfers
- fare
        ↓
Passenger boards
        ↓
Passenger identifies themselves
        ↓
Vehicle is identified
        ↓
Journey begins
        ↓
Vehicle/passenger movement is tracked
        ↓
Passenger approaches destination
        ↓
Passenger receives reminder
        ↓
Passenger gets off
        ↓
System determines alighting
        ↓
Fare is calculated
        ↓
Payment request sent
        ↓
Existing payment system processes transaction
        ↓
Journey marked COMPLETED
        ↓
Passenger sees journey/payment record
```

If this complete flow works, the core product has been built.

Additional features after this point are **enhancements**, not reasons to keep the project permanently unfinished.

---

30. Future Features

Future versions may include:

- live arrival predictions;

- real-time vehicle positions for passengers;
- traffic-aware journey estimates;
- operator analytics;
- conductor analytics;
- fleet management;
- route performance analytics;
- passenger demand analysis;
- mobile-money integration;
- multiple cities;
- advanced offline navigation;
- automatic route disruption detection;
- accessibility features;
- multilingual interface;
- operator revenue dashboards.

These are explicitly outside the first completion target.

---

31. Final Product Boundary

The first complete product is:

A Dar es Salaam passenger journey platform that knows the transport network, helps passengers find and follow daladala journeys, reminds them when they approach their destination, determines their journey fare, and can integrate that fare with an existing digital payment system.

The project should not expand indefinitely before this core flow is operational.

The build should proceed phase-by-phase.

Each phase must have:

1. defined functionality;
2. implementation;
3. tests;
4. demonstration;
5. Definition of Done.

Only after a phase is complete should the next phase begin.